Buổi 07 (3 tiết) – Chương 3: Vòng đời, Controlled Component, Lồng component (3.4–3.6)

1.1 3.4 Trình tự hoạt động của một Component

Một component React Native trải qua một chu trình hoạt động (lifecycle) bao gồm các giai đoạn chính sau:

1. **Mounting (Gắn kết):** Đây là giai đoạn component được tạo và chèn vào DOM (ảo).
    - **Constructor (chỉ dành cho Class Component):** Được gọi đầu tiên khi component được tạo. Thường được sử dụng để khởi tạo state và binding các phương thức xử lý sự kiện.
    - **render():** Phương thức quan trọng nhất, trả về JSX mô tả những gì sẽ hiển thị trên màn hình.
    - **componentDidMount() (chỉ dành cho Class Component):** Được gọi ngay sau khi component được render lần đầu tiên và đã được thêm vào DOM. Đây là nơi thích hợp để thực hiện các side effect như gọi API hoặc thiết lập timers.
    - **useEffect với dependency array rỗng ([]) (cho Functional Component):** Tương tự như componentDidMount, effect này chỉ chạy một lần sau lần render đầu tiên.

2. **Updating (Cập nhật):** Giai đoạn component được render lại do sự thay đổi của props hoặc state.
    - **shouldComponentUpdate(nextProps, nextState) (chỉ dành cho Class Component):** Được gọi trước khi quá trình render lại bắt đầu. Bạn có thể trả về false để ngăn việc render lại nếu không cần thiết, giúp tối ưu hóa hiệu suất.
    - **render():** Được gọi lại khi props hoặc state thay đổi.
    - **componentDidUpdate(prevProps, prevState, snapshot) (chỉ dành cho Class Component):** Được gọi sau khi component được render lại. Bạn có thể thực hiện các side effect dựa trên sự thay đổi của propshoặc state.
    - **useEffect với dependency array (cho Functional Component):** Effect này sẽ chạy lại mỗi khi bất kỳ giá trị nào trong dependency array thay đổi.
3. **Unmounting (Tháo gỡ):** Giai đoạn component bị xóa khỏi DOM.
    - **componentWillUnmount() (chỉ dành cho Class Component):** Được gọi ngay trước khi component bị xóa. Đây là nơi thích hợp để dọn dẹp các tài nguyên đã sử dụng trong componentDidMount hoặc componentDidUpdate (ví dụ: hủy bỏ subscriptions, xóa timers).
    - **Cleanup function trong useEffect (cho Functional Component):** Hàm được trả về từ useEffect sẽ được gọi trước khi effect chạy lại lần tiếp theo hoặc khi component bị unmount.

1.2 3.5 Các Component điều khiển

Trong React Native, một **Controlled Component** là một component mà dữ liệu của nó (thường là giá trị của các input như TextInput) được kiểm soát và quản lý bởi state của React. Khi người dùng tương tác với input (ví dụ: gõ chữ), một hàm xử lý sự kiện (event handler) được gọi để cập nhật state, và state được cập nhật sẽ làm cho component render lại với giá trị mới.

Đặc điểm của Controlled Component:

- Giá trị của input luôn được đồng bộ với state của component.
- Mọi thay đổi của input đều được điều khiển bởi code React.
- Dễ dàng thực hiện các logic nghiệp vụ dựa trên giá trị của input (ví dụ: validation).

Ví dụ:

``javascript import React, { useState } from 'react'; import { View, TextInput, Text, StyleSheet } from 'react-native';

const ControlledInput = () => { const [text, setText] = useState('');

const handleChangeText = (newText) => { setText(newText); };

return (<View style={styles.container}> <TextInput style={styles.input} value={text} onChangeText={handleChangeText} placeholder="Nhập gì đó..." /> <Text style={styles.text}>Bạn đã nhập: {text}</Text> </View>); };

const styles = StyleSheet.create({ container: { padding: 20, }, input: { height: 40, borderColor: 'gray', borderWidth: 1, marginBottom: 10, paddingHorizontal: 10, }, text: { fontSize: 16, }, });

export default ControlledInput; ``

Trong ví dụ này:

- useState('') tạo ra một state variable text và hàm setText để cập nhật nó.
- Thuộc tính value của TextInput được gán bằng giá trị của text.
- Thuộc tính onChangeText nhận một hàm handleChangeText, hàm này được gọi mỗi khi văn bản trong TextInput thay đổi. Hàm này cập nhật state text bằng giá trị mới.
- Khi state text thay đổi, component ControlledInput được render lại và TextInput hiển thị giá trị mới.

1.3 3.6 Component lồng Component

Một trong những sức mạnh của React Native là khả năng tạo ra các giao diện phức tạp bằng cách lồng các component lại với nhau. Một component có thể chứa một hoặc nhiều component khác bên trong phần JSX của nó.

Ví dụ:

Giả sử bạn có hai component: UserProfile và Avatar. Bạn có thể lồng Avatar vào bên trong UserProfile để hiển thị ảnh đại diện của người dùng.

Avatar Component:

``javascript import React from 'react'; import { Image, StyleSheet } from 'react-native';

const Avatar = (props) => { return (<Image style={styles.avatar} source={{ uri: props.imageUrl }} />); };

const styles = StyleSheet.create({ avatar: { width: 50, height: 50, borderRadius: 25, }, });

export default Avatar; ``

UserProfile Component:

``javascript import React from 'react'; import { View, Text, StyleSheet } from 'react-native'; import Avatar from './Avatar';

const UserProfile = (props) => { return (<View style={styles.container}> <Avatar imageUrl={props.profileImage} /> <Text style={styles.name}>{props.name}</Text> <Text style={styles.bio}>{props.bio}</Text> </View>); };

const styles = StyleSheet.create({ container: { alignItems: 'center', padding: 20, }, name: { fontSize: 18, fontWeight: 'bold', marginTop: 10, }, bio: { textAlign: 'center', marginTop: 5, }, });

export default UserProfile; ``

Trong UserProfile, chúng ta import và sử dụng component Avatar bên trong phần return. Chúng ta cũng truyền props (profileImage) cho component Avatar. Khi sử dụng UserProfile ở một nơi khác, bạn sẽ truyền dữ liệu người dùng vào, bao gồm cả đường dẫn ảnh đại diện.

Cách sử dụng UserProfile:

``javascript import React from 'react'; import { ScrollView } from 'react-native'; import UserProfile from './UserProfile';

const App = () => { return (<ScrollView> <UserProfile name="Nguyễn Văn A" bio="Lập trình viên React Native" profileImage="[liên kết đáng ngờ đã bị xóa]" /> <UserProfile name="Trần Thị B" bio="Nhà thiết kế UI/UX" profileImage="[liên kết đáng ngờ đã bị xóa]" /> </ScrollView>); };

export default App; ``